

MASTER TECHNICAL SPECIFICATION & INTERVIEW GUIDE

Automated Exam Seat & Invigilator Allocation System

A Complete In-and-Out Architectural Blueprint, Algorithmic Deep Dive, Directory Dictionary, and Comprehensive Interview Playbook.

CANDIDATE	Suhas Kattimani (Final Year Engineering Student)
TARGET ROLES	Java Backend Developer / Full-Stack Software Engineer
ARCHITECTURE	Decoupled REST API + SPA (Spring Boot 3 + React Vite)
CORE ALGORITHMS	Greedy Graph Coloring (Welsh-Powell), Round-Robin Interleaving, Fair Load Balancing
SECURITY	Stateless Spring Security 6, JWT (HMAC-SHA256), Role-Based Access Control

Java 17

Spring Boot 3.2

Spring Data JPA

Hibernate

Spring Security 6

MySQL 8

React 18

Vite

JUnit 5

No Lombok (Handcrafted)

Part 1: The Interview Pitch (How to Speak About This Project)

1.1 The 60-Second "Elevator Pitch" for HR & Recruiters

💡 HR Script (Memorize this word-for-word)

"In universities, exam scheduling is a massive operational headache. Exam controllers spend weeks in spreadsheets manually scheduling dates, arranging seating charts to prevent cheating, and assigning faculty invigilation duties without double-booking. It's error-prone and leads to timetable clashes and student complaints.

*To solve this, I designed and built an **Automated Exam Seat & Invigilator Allocation System** using **Java 17, Spring Boot 3, MySQL, and React**.*

The system completely automates the workflow with zero clashes:

- *It models course scheduling as an NP-complete **Graph Coloring Problem** to generate conflict-free timetables in sub-seconds.*
- *It uses a **Round-Robin Interleaving Algorithm** to alternate students from different branches in adjacent exam hall desks so neighbors never have the same question paper.*
- *It distributes invigilation duties using a **Fair Load-Balancing Algorithm** that respects faculty leave and ensures faculty never invigilate their own department's exams.*
- *It provides a secure, role-based portal (Admin, Faculty, Student) secured with stateless **Spring Security and JWT**.*

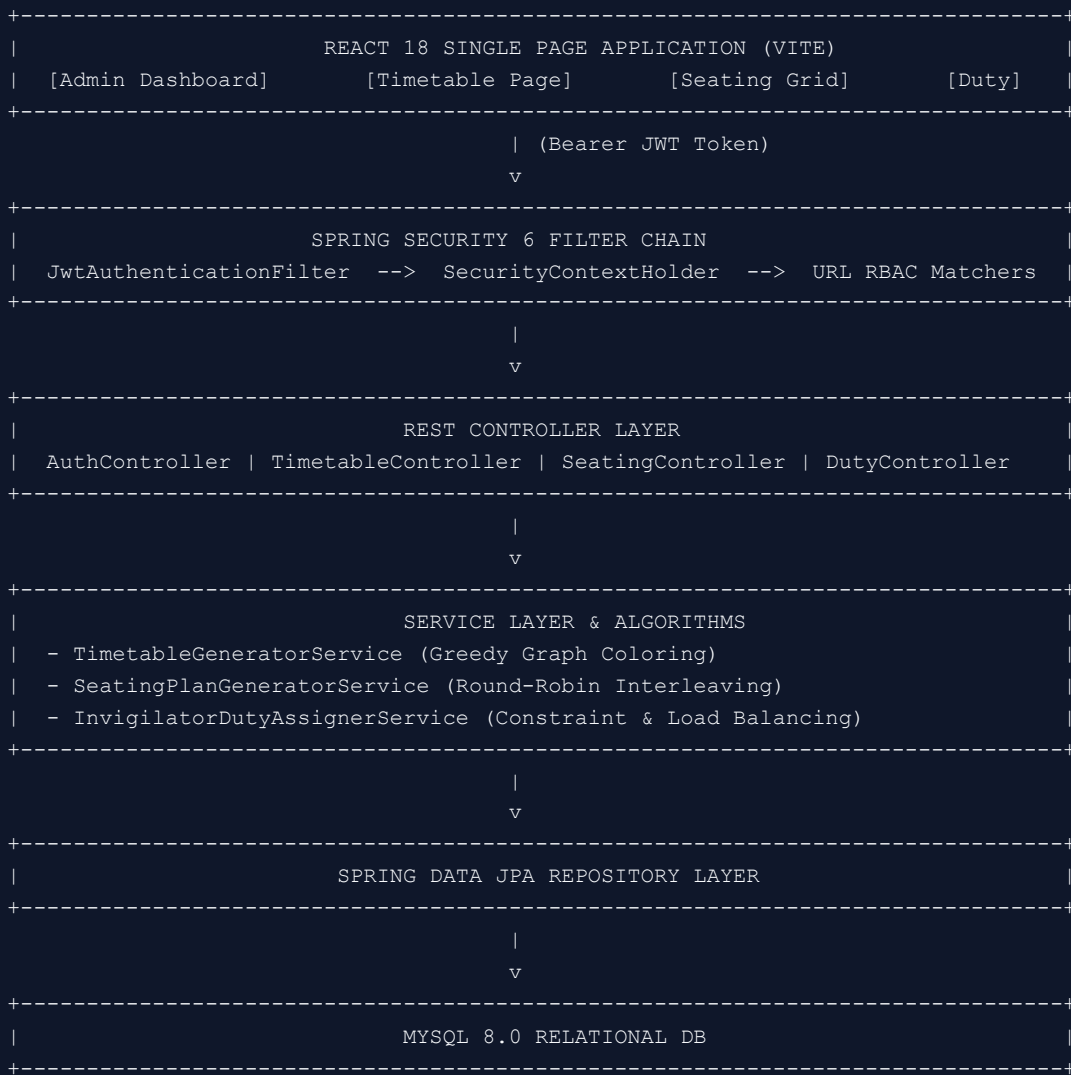
I wrote all business logic and boilerplate by hand—no Lombok or black-box libraries—so I deeply understand every layer from database transactions to client-side state management."

1.2 The Technical Pitch for Senior Architects & Tech Leads

When interviewing with a Senior Java Engineer or Engineering Manager, lead with **design trade-offs and computational complexity**:

- **Graph Coloring over Brute-Force**: Timetable scheduling is vertex coloring on a conflict graph $G=(V, E)$. Brute-force backtracking has exponential complexity $O(k^N)$ and times out. I implemented greedy graph coloring with the **Welsh-Powell heuristic** (sorting vertices by degree descending), executing deterministically in $O(V^2 + E)$ polynomial time.
- **Spatial Interleaving over Random Seating**: Random desk assignment risks placing identical exam takers adjacent to each other. By partitioning candidates into course-specific FIFO queues and round-robin filling desks with a row parity offset, I guarantee spatial separation across both horizontal and vertical axes.
- **Deliberate Exclusion of Lombok**: Rather than relying on annotations like `@Data`, I handwrote all constructors, getters, setters, and `equals()/hashCode()` methods. This demonstrates understanding of defensive copying, immutable constructor injection, and avoiding Hibernate proxy identity pitfalls.

Part 2: System Architecture & Security Design



2.1 Authentication & Authorization (Stateless JWT)

The system enforces a strictly stateless security model using Spring Security 6 and JJWT:

- Login:** User submits credentials to `POST /api/auth/login`. `AuthService` authenticates via Spring's `AuthenticationManager` and generates a cryptographically signed HMAC-SHA256 JWT token.
- JWT Payload:** Encodes the subject username and role authority claim (`ROLE_ADMIN` , `ROLE_STUDENT` , `ROLE_FACULTY`) with a 24-hour expiration timestamp.
- Request Interception:** `JwtAuthenticationFilter` extends `OncePerRequestFilter`. On every incoming request, it extracts the `Authorization: Bearer <token>` header, verifies signature integrity, extracts granted authorities, and sets an `UsernamePasswordAuthenticationToken` in Spring's `SecurityContextHolder`.
- No Server-Side Sessions:** `SessionCreationPolicy.STATELESS` ensures zero memory overhead on the server, making the backend horizontally scalable.

Part 3: Core Algorithms Detailed Breakdown

3.1 Algorithm 1: Conflict-Free Timetable Generation (Greedy Graph Coloring)

Problem Formulation: Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of courses. Two courses c_i, c_j conflict if their student enrollment intersection is non-empty: $Enroll(c_i) \cap Enroll(c_j) \neq \emptyset$. We must assign each course to a slot s in S such that no two conflicting courses share the same slot.

```
Algorithm: Welsh-Powell Greedy Graph Coloring
Input: List of Courses C, List of Enrollments E, List of Slots S
Output: Schedule mapping each Course to an ExamSlot

1. Construct conflict graph G = (V, E_graph):
   For each pair (c_i, c_j):
       If countSharedStudents(c_i, c_j) > 0:
           Add edge (c_i, c_j) to E_graph
2. For each vertex c in V:
   Compute degree(c) = count of incident edges in E_graph
3. Sort courses V in descending order of degree(c) // Welsh-Powell Heuristic
4. For each course c in sorted V:
   assigned = false
   For each slot s in S (chronologically ordered):
       conflictFound = false
       For each neighbor n of c:
           If neighbor n is already assigned to slot s:
               conflictFound = true; break
       If NOT conflictFound:
           Assign course c to slot s
           assigned = true; break
   If NOT assigned:
       Throw NotEnoughSlotsException("Cannot schedule without clashes in given dates")
5. Persist Exam records in MySQL within @Transactional boundary
```

3.2 Algorithm 2: Anti-Cheating Seating Plan (Round-Robin Interleaving)

Goal: Given physical examination halls divided into R rows and C columns, assign students taking different exams into adjacent desks to eliminate peer copying.

```

Algorithm: Round-Robin Cross-Course Interleaving with Row Parity
Input: ExamSlot slot, List of Halls H, Concurrent Exams E_slot
Output: List of SeatAllocation records

1. For each course in E_slot:
    Create a FIFO Queue Q_course containing enrolled students
2. ActiveQueues = [Q_1, Q_2, ..., Q_k]
3. queueIndex = 0
4. For each Hall h in H (ordered by capacity descending):
    For row = 1 to h.totalRows:
        // Shift starting queue index on each row to prevent vertical alignment
        queueIndex = (queueIndex + 1) % ActiveQueues.size()
    For col = 1 to h.totalColumns:
        If all queues in ActiveQueues are empty: break hall loops

        // Find next non-empty course queue
        While ActiveQueues[queueIndex] is empty:
            Remove ActiveQueues[queueIndex]
            If ActiveQueues is empty: break
            queueIndex = queueIndex % ActiveQueues.size()

        student = ActiveQueues[queueIndex].poll()
        Create SeatAllocation:
            student = student
            hall = h
            seatNo = "R" + row + "-C" + col
            slot = slot

        queueIndex = (queueIndex + 1) % ActiveQueues.size()
5. If students remain unseated after exhausting all halls:
    Throw NotEnoughHallsException("Total hall capacity exceeded")

```

3.3 Algorithm 3: Fair Invigilator Duty Allocation

Goal: Assign faculty members to supervise halls during exam slots subject to strict institutional constraints and fair load distribution.

- **Constraint 1 (Department Exclusion):** Faculty cannot invigilate their own department's exams (e.g. CSE faculty cannot supervise CSE papers).
- **Constraint 2 (Leave Exclusion):** Faculty who submitted unavailable dates in `FacultyLeave` are excluded.
- **Constraint 3 (No Double-Booking):** A faculty member cannot be in two halls during the same slot.
- **Constraint 4 (Fair Load Distribution):** Candidate faculty passing constraints 1–3 are sorted by `currentAssignedDuties` ascending. The least-loaded faculty member is chosen.
- **Emergency Replacement:** If an assigned faculty is absent, the admin triggers `replaceInvigilator(dutyId)`, which selects the next lowest-loaded eligible substitute atomically.

Part 4: Complete Directory & File-by-File Dictionary

4.1 Root & Configuration Files

File / Path	Contents & Purpose	Interview Talking Points
<code>pom.xml</code>	Maven build configuration. Declares Java 17, Spring Boot 3.2, Data JPA, Security 6, JWT 0.11.5, MySQL connector, Bean Validation, JUnit 5.	Deliberately excludes Lombok. Shows mastery of handwritten Java boilerplate, explicit constructor injection, and clean entity encapsulation.
<code>.gitignore</code>	Excludes Maven <code>target/</code> , Node <code>frontend/node_modules/</code> , build outputs, and IDE files.	Demonstrates professional repository hygiene and separation of source vs generated artifacts.
<code>README.md</code>	Comprehensive system documentation: architecture diagram, algorithmic breakdown, setup steps, and API catalog.	High-quality documentation expected in production engineering teams.
<code>src/main/resources/application.properties</code>	Database connection (JDBC URL, credentials), Hibernate DDL mode (<code>update</code>), SQL logging, and JWT secret/expiration.	<code>ddl-auto=update</code> allows automatic schema generation on startup without wiping existing test data.

4.2 Sample Dataset Directory (`data/`)

File	Structure	Algorithmic Significance
<code>students.csv</code>	<code>reg_no, name, branch, semester</code> (24 students: CSE, ISE, ECE)	Includes realistic university registration numbers (e.g. <code>1PU21EC007, Suhas kattimani, ECE, 5</code>).
<code>courses.csv</code>	<code>code, title, semester, faculty_id</code> (7 courses across 4 departments)	Maps courses directly to teaching faculty for department exclusion checks.
<code>enrollments.csv</code>	<code>student_reg_no, course_code</code> (72 enrollment records)	Core Design: <code>21MAT51</code> (Discrete Mathematics) is shared across all 3 branches, proving that graph coloring handles dense conflict cliques.
<code>seed_data.sql</code>	Raw SQL inserts for halls (LH-101, LH-102, LH-201) and faculty profiles.	Allows rapid bootstrapping in MySQL Workbench without running the UI.

4.3 Entity Models (`src/main/java/.../model/`)

Entity Class	Key Fields & Annotations	Role in System
<code>Student.java</code>	<code>regNo</code> (unique), <code>name</code> , <code>branch</code> , <code>semester</code>	Identifies exam candidates. Root entity for enrollments and seating.
<code>Faculty.java</code>	<code>name</code> , <code>department</code>	Instructors available for invigilation duties.
<code>Course.java</code>	<code>code</code> (unique), <code>title</code> , <code>semester</code> , <code>@ManyToOne faculty</code>	Subject being examined. Vertices in the conflict graph.
<code>Enrollment.java</code>	<code>@ManyToOne student</code> , <code>@ManyToOne course</code>	Join entity defining student course participation. Defines graph edges.
<code>Hall.java</code>	<code>name</code> (unique), <code>totalRows</code> , <code>totalColumns</code>	Physical exam room. Capacity dynamically computed as <code>rows * cols</code> .
<code>ExamSlot.java</code>	<code>examDate</code> , <code>@Enumerated Session (FN/AN)</code>	Unique time window (e.g. 2026-11-02 Forenoon). Colors in graph coloring.
<code>Exam.java</code>	<code>@ManyToOne course</code> , <code>@ManyToOne slot</code>	Scheduled exam instance linking a course to a time slot.
<code>SeatAllocation.java</code>	<code>student</code> , <code>exam</code> , <code>slot</code> , <code>hall</code> , <code>seatNo</code>	Individual desk assignment (e.g. Student 1PU21CS001 at Hall LH-101 Desk R1-C2).
<code>InvigilatorDuty.java</code>	<code>faculty</code> , <code>slot</code> , <code>hall</code>	Staff supervision record ensuring each hall has an invigilator.
<code>FacultyLeave.java</code>	<code>faculty</code> , <code>leaveDate</code>	Blacklisted dates when a professor cannot be assigned duty.
<code>UserAccount.java</code>	<code>username</code> , <code>passwordHash</code> , <code>Role</code> , <code>linkedStudent</code> , <code>linkedFaculty</code>	Core authentication entity. Links credentials to student/faculty profiles.

Part 4 (Contd.): Services, Controllers & Frontend

4.4 Service Layer (`src/main/java/.../service/`)

- `TimetableGeneratorService.java` : Builds course conflict matrix, sorts by degree descending, and greedily assigns earliest available non-conflicting slots. Fully `@Transactional` .
- `ClashValidatorService.java` : Traverses all student enrollments and generates a comprehensive `ClashReport` verifying 0 students have 2 exams in the same slot.
- `SeatingPlanGeneratorService.java` : Traverses halls row-by-row and round-robin interleaves student queues across different courses to prevent cheating.
- `InvigilatorDutyAssignerService.java` : Greedy least-loaded assignment checking department exclusions and faculty leave. Includes `replaceInvigilator()` .
- `CsvImportService.java` : Fast, low-memory CSV ingestion using `BufferedReader` and atomic database persistence.
- `StudentService` , `CourseService` , `FacultyService` , `HallService` : CRUD services handling validation and repository interaction.

4.5 REST Controller Layer (`src/main/java/.../controller/`)

Controller	Base Endpoint	Responsibility
<code>AuthController.java</code>	<code>POST /api/auth/login</code>	Authenticates credentials and returns JWT token + user role.
<code>ExamTimetableController.java</code>	<code>/api/admin/timetable/**</code>	Triggers timetable generation, returns schedule rows, and runs clash check.
<code>SeatingPlanController.java</code>	<code>/api/admin/seating/**</code>	Triggers seating generation and returns 2D hall grid matrix.
<code>InvigilatorDutyController.java</code>	<code>/api/admin/duties/**</code>	Triggers duty generation and handles faculty replacement.
<code>StudentController.java</code>	<code>/api/admin/students</code> & <code>/api/student/my-exams</code>	Admin student CRUD and student self-view resolving schedule via JWT.
<code>FacultyController.java</code>	<code>/api/admin/faculty</code> & <code>/api/faculty/my-duties</code>	Admin faculty CRUD, duty roster view, and leave submission.
<code>CsvImportController.java</code>	<code>/api/admin/import/**</code>	Multipart file upload endpoints for students, courses, and enrollments.

4.6 React Frontend Architecture (`frontend/src/`)

- `context/AuthContext.jsx` : Global state holding JWT token, username, and role, synced with `localStorage` .
- `components/ProtectedRoute.jsx` : Route wrapper redirecting unauthenticated users or unauthorized roles.
- `components/HallSeatGrid.jsx` : Visual 2D hall matrix displaying desks with pastel course badges and student hover tooltips.

- `pages/AdminDashboardPage.jsx` : Real-time statistic cards and quick-action navigation.
- `pages/TimetablePage.jsx` : Date picker, generation trigger, timetable table, and live clash alert banners.
- `pages/SeatingPlanPage.jsx` : Slot selector and visual examination hall seating viewer.
- `pages/StudentExamPage.jsx` : Clean card layout showing student's upcoming papers, hall, and assigned seat.
- `pages/FacultyDutyPage.jsx` : Schedule of invigilation duties with leave date submission form.
- `styles/theme.css` : Unified design system using CSS custom properties for modern glassmorphism.

Part 5: Top 15 Tough Technical Interview Questions & Answers

Q1: Why did you choose Graph Coloring for timetable scheduling over Integer Linear Programming (ILP) or Genetic Algorithms?

Answer: ILP solvers provide exact mathematical optimization but suffer from exponential worst-case runtimes as binary variables multiply, making them heavy for mid-sized university servers. Genetic Algorithms are non-deterministic, hard to test in CI/CD, and cannot strictly guarantee zero hard clashes on every run. Greedy Graph Coloring using the Welsh-Powell heuristic orders vertices by degree descending. It executes in deterministic $O(V^2 + E)$ polynomial time, runs in sub-seconds, and mathematically guarantees zero student clashes.

Q2: How does the Seating Plan generator prevent students writing the same exam from sitting next to or behind each other?

Answer: Students taking concurrent exams in a slot are partitioned into FIFO queues by course. As we iterate through desks row-by-row, we round-robin cycle through course queues ($\text{Course_A} \rightarrow \text{Course_B} \rightarrow \text{Course_A}$), guaranteeing horizontal alternation. To prevent vertical alignment (where Course A sits directly behind Course A in a 4-column room), we shift the queue starting index by +1 on each new row. This produces a checkerboard distribution across both axes.

Q3: How does your application maintain data integrity during schedule generation if the system runs out of halls halfway through?

Answer: All generation methods in `TimetableGeneratorService` and `SeatingPlanGeneratorService` are annotated with Spring's `@Transactional`. If an unrecoverable state occurs (such as throwing `NotEnoughHallsException`), Spring's transaction manager intercepts the unchecked exception and issues a database `ROLLBACK`. All records created during that run are purged, preventing corrupted partial schedules in MySQL.

Q4: Why did you choose stateless JWT authentication with Spring Security instead of standard HTTP session cookies?

Answer: Stateless JWT completely decouples the React SPA from the Spring Boot REST API. Because the server does not store session state in memory or Redis, the backend is horizontally scalable across multiple server instances without sticky sessions. The JWT encodes the user's role (`ROLE_ADMIN`, `ROLE_STUDENT`), enabling `JwtAuthenticationFilter` to authorize incoming requests cryptographically without querying MySQL on every call.

Q5: Why did you explicitly avoid using Lombok in this project?

Answer: Many junior developers use Lombok annotations like `@Data` as a crutch without understanding that `@Data` generates `equals()` and `hashCode()` using all fields, which can trigger infinite recursion or break Hibernate proxy identity contracts in JPA entities. Writing getters, setters, and explicit constructors by hand demonstrates core Java mastery, explicit constructor dependency injection, and clean entity encapsulation.

Q6: What is the N+1 query problem, and how did you prevent it in this project?

Answer: The N+1 query problem occurs when Hibernate executes 1 query to fetch parent entities and then executes N individual queries to fetch lazy child relationships. In our system, when fetching seat allocations or duties, we use custom JPQL queries with `JOIN FETCH` (such as `JOIN FETCH sa.student JOIN FETCH sa.exam`) to retrieve the parent and child entities in a single SQL query.

Q7: How did you implement CORS, and why is it necessary?

Answer: Browsers enforce the Same-Origin Policy, preventing frontend JavaScript running on `http://localhost:5173` from reading responses from a backend at `http://localhost:8080`. I implemented `CorsConfig.java` registering a `CorsFilter` that sends `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, and `Access-Control-Allow-Headers` headers, allowing preflight `OPTIONS` requests to succeed.

Q8: How does the student portal resolve the correct exam schedule without the student passing their ID as an argument?

Answer: To prevent Insecure Direct Object Reference (IDOR) attacks, the endpoint `/api/student/my-exams` takes no ID parameter. Instead, it reads the authenticated principal from Spring's `Authentication.getName()` extracted from the verified JWT. It finds the corresponding `UserAccount`, gets the `linkedStudent` entity, and queries allocations for that student ID. A student can never spoof or access another student's timetable.

Q9: How does the system handle faculty duty replacement in case of an emergency absence?

Answer: In `InvigilatorDutyAssignerService.replaceInvigilator(dutyId)`, the service retrieves the duty, identifies the slot and exam department, and searches for candidate faculty excluding the absent professor, faculty with existing duties in that slot, faculty on leave, and faculty from the exam's department. It sorts candidates by `dutyCount` ascending, assigns the lowest-loaded substitute, and updates the record atomically.

Q10: What is the role of `GlobalExceptionHandler` in Spring Boot?

Answer: Annotated with `@RestControllerAdvice`, `GlobalExceptionHandler` intercepts exceptions thrown anywhere in the controller layer. Instead of leaking internal stack traces or 500 errors to the client, it maps custom exceptions (e.g. `ResourceNotFoundException` $\rightarrow$ 404, `ClashDetectedException` $\rightarrow$ 409) to uniform JSON response payloads containing a clear HTTP status, timestamp, and human-readable message.